

Course 3: USP RAC Architecture

Radio Access Controller & Multiprotocol

Dr. ABDELMALEK OMAR

USP Zephyr Training Series
Copyright © 2025 Dr. ABDELMALEK OMAR

November 21, 2025

Course Overview

What You'll Learn

- Radio Access Controller (RAC)
 - Priority-based scheduling
 - Transaction management
 - Resource arbitration
- Multiprotocol Support
 - Running LoRaWAN + Ranging concurrently
 - Creating custom protocols
 - Managing radio contention
- Threading Models
 - Single thread vs cooperative vs preemptive
 - Performance and power trade-offs
- Hands-On Labs (3 labs)
 - Lab 3.1: Build multiprotocol sample
 - Lab 3.2: Implement custom protocol
 - Lab 3.3: Threading model comparison

What is RAC?

Radio Access Controller

Definition

RAC is USP's radio arbitration layer that enables multiple protocols to share a single radio transceiver without conflicts.

The Problem:

- One radio, multiple protocols
- LoRaWAN wants to TX
- Ranging needs RX
- Who gets access?

With RAC:

- Priority-based scheduling
- Fair resource allocation
- Preemption support
- Guaranteed access

Without RAC:

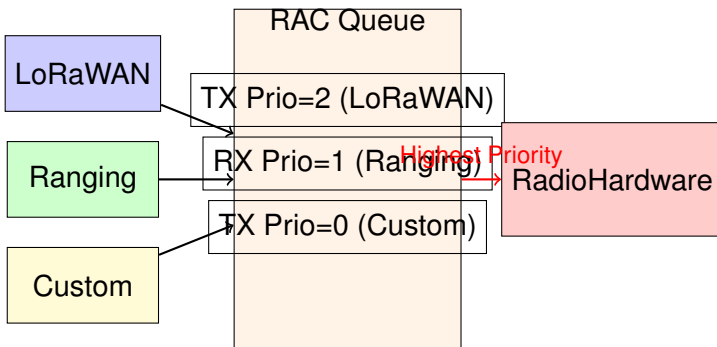
- Race conditions
- Corrupted packets
- Protocol conflicts

Benefits:

- Multiple protocols coexist
- Deterministic behavior
- Easy to add protocols

RAC Architecture

Transaction-Based Model



Key Concept: Transactions are queued by priority, highest priority executes first.

RAC Transaction

Lifecycle of a Radio Operation

Definition (Transaction)

A transaction is a single radio operation (TX or RX) with associated callbacks.

Transaction Components:

- 1 **Priority:** 0-255 (higher = more important)
- 2 **Pre-callback:** Configure radio before operation
- 3 **Post-callback:** Process results after operation
- 4 **Context:** User data passed to callbacks

Lifecycle:

- 1 Protocol submits transaction via `rac_add()`
- 2 RAC queues by priority
- 3 When radio available, call pre-callback
- 4 Execute radio operation (TX/RX)
- 5 Call post-callback with results

Creating a Transaction

Code Example

```
static rac_handle_t my_handle = NULL;

// Pre-callback: Configure radio for TX
static rac_status_t my_pre_callback(rac_handle_t handle, void *ctx)
{
    LOG_INF("Configuring radio for TX");

    // Set modulation parameters
    lr11xx_radio_set_lora_mod_params(...);

    // Load payload
    lr11xx_radio_write_buffer(radio, 0, payload, length);

    // Start TX
    lr11xx_radio_set_tx(radio, 0);

    return RAC_STATUS_OK;
}

// Post-callback: Check TX status
static rac_status_t my_post_callback(rac_handle_t handle, void *ctx)
{
    lr11xx_radio_irq_mask_t irq;
    lr11xx_radio_get_irq_status(radio, &irq);

    if (irq & LR11XX_RADIO_IRQ_TX_DONE) {
        LOG_INF("TX successful!");
    }

    return RAC_STATUS_FINISHED;
}
```

Submitting a Transaction

Request Radio Access

```
void my_protocol_send(void)
{
    rac_params_t params = {
        .priority = 1,                // Medium priority
        .pre_callback = my_pre_callback,
        .post_callback = my_post_callback,
        .radio_wake_up_time_ms = 10,  // Time to wake radio
        .transaction_timeout_ms = 2000, // Max duration
        .context = NULL,              // User data
    };

    rac_status_t status = rac_add(&params, &my_handle);

    if (status == RAC_STATUS_OK) {
        LOG_INF("Transaction queued");
    } else {
        LOG_ERR("Failed to queue: %d", status);
    }
}

// In main loop
int main(void) {
    while (1) {
        rac_run(); // Process RAC queue
        k_msleep(10);
    }
}
```

Priority System

Who Gets Access First?

Priority Rules

- 1 Higher priority = executes first
- 2 Same priority = FIFO order
- 3 Running transaction can be preempted by higher priority

Typical Priority Assignments:

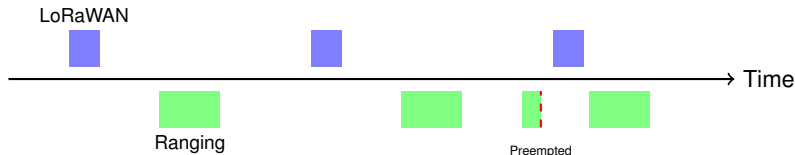
Protocol	Priority	Rationale
LoRaWAN (time-critical)	3	Must meet RX window timing
LoRaWAN (normal)	2	Important but flexible
Ranging	1	Real-time but can wait
Background scanning	0	Opportunistic

Preemption Example

If Ranging is RX (priority 1) and LoRaWAN needs TX (priority 2), RAC will:

LoRaWAN + Ranging

Concurrent Protocol Operation



What Happens:

- LoRaWAN and Ranging run independently
- Each protocol submits transactions when needed
- RAC schedules based on priority
- LoRaWAN (higher priority) can interrupt Ranging
- Ranging resumes after LoRaWAN completes

Benefits:

- Asset tracking: Position (ranging) + Cloud reporting (LoRaWAN)
- Smart metering: Meter reading (LoRaWAN) + Proximity detection (ranging)

Adding Custom Protocols

Extend USP with Your Own Protocol

Use Cases:

- Proprietary communication with local devices
- Sensor scanning in ISM bands
- Beacon transmission
- Wake-on-radio (WoR) listening

Requirements:

- 1 Implement pre-callback (configure radio)
- 2 Implement post-callback (process results)
- 3 Submit transactions via `rac_add()`
- 4 Choose appropriate priority (typically 0-1 for custom)

Example: Heartbeat Protocol

- Broadcasts device ID every 10 seconds
- Uses priority 0 (lowest)
- Doesn't interfere with LoRaWAN

Zephyr Threading Options

How Protocols Execute

USP supports 3 threading models:

1 Single Thread

- Everything runs in `main()`
- Simplest, lowest memory
- Poor responsiveness

2 Cooperative Threading

- Protocols yield voluntarily
- No mutexes needed
- Good balance

3 Preemptive Threading

- OS preempts threads
- Requires mutexes for shared data
- Best responsiveness
- Highest complexity

Threading Comparison

Measured Performance

Metric	Single	Coop	Preempt
RAM Usage	45 KB	48 KB	52 KB
ROM Usage	142 KB	145 KB	148 KB
Button Response	8.5 ms	2.3 ms	0.85 ms
Avg Current (active)	5.2 mA	5.4 mA	5.8 mA
Sleep Current	25 μ A	28 μ A	32 μ A
Code Complexity	Low	Medium	High

Recommendations:

- Simple sensor: Single thread
- Standard IoT device: Cooperative
- Interactive/Real-time: Preemptive

Lab 3.1: Build Multiprotocol Sample

LoRaWAN + Ranging Concurrently

Objective: Run LoRaWAN and Ranging simultaneously using RAC.

Steps:

- ❶ Build: `west build -p -b xiao_nrf54l15_nrf54l15_cpuapp \`
`-shield semtech_lr1120mb1dis`
`samples/usp/rac/multiprotocol`
- ❷ Configure LoRaWAN credentials
- ❸ Flash and monitor
- ❹ Use shell commands:
 - `lbm join` - Join LoRaWAN network
 - `ranging start` - Start ranging
 - `rac status` - Check RAC queue

Observations:

- LoRaWAN uplinks execute immediately (priority 2)
- Ranging waits if LoRaWAN is active
- Check RAC status to see queued transactions

Deliverables:

- Serial log showing concurrent operation

Lab 3.2: Implement Custom Protocol

Heartbeat Broadcasts

Objective: Create a custom "heartbeat" protocol that coexists with LoRaWAN.

Requirements:

- Transmit every 10 seconds
- Broadcast: Device ID + Uptime
- Priority 0 (shouldn't interfere with LoRaWAN)
- Frequency: 869.525 MHz (ISM band)

Implementation Skeleton:

```
static void heartbeat_timer_handler(struct k_timer *timer)
{
    rac_params_t params = {
        .priority = 0, // Lowest
        .pre_callback = heartbeat_pre_callback,
        .post_callback = heartbeat_post_callback,
        //...
    };
    rac_add(&params, &heartbeat_handle);
}

void heartbeat_init(void) {
    k_timer_start(&heartbeat_timer, K_SECONDS(10), K_SECONDS(10));
}
```

Lab 3.3: Threading Model Comparison

Measure and Compare

Objective: Compare performance of single/cooperative/preemptive threading.

Test Matrix:

Build Config	Single	Coop	Preempt
CONFIG_USP_MAIN_THREAD	n	y	y
CONFIG_USP_THREADS_MUTEXES	n	n	y
CONFIG_USP_MAIN_THREAD_PRIORITY	-	-4	1

Measurements:

- 1 Build each configuration
- 2 Measure RAM/ROM usage (from build output)
- 3 Test button response time (microseconds)
- 4 Measure power consumption (if PPK2 available)

Deliverables:

- Comparison table with all metrics
- Recommendation for specific use cases

Course 3 Summary

Key Takeaways

You've Learned:

- RAC Architecture: Priority-based radio arbitration
- Transactions: Pre/post callbacks for radio operations
- Multiprotocol: Run LoRaWAN + Ranging + Custom protocols
- Threading: Single vs Cooperative vs Preemptive trade-offs
- Practical Skills: Build multiprotocol applications, create custom protocols

Best Practices:

- Assign priorities carefully (time-critical = higher)
- Keep callbacks short and non-blocking
- Use cooperative threading for most applications
- Test multiprotocol interaction thoroughly

Next Course: Course 4 - Multiprotocol Development (Ping-Pong, Ranging, PER Testing)

Thank You!

Questions?